End-to-End Correctness and Corner Cases

> ***Purpose.*** *This is a cross-subsystem study report showing how to reason about the complete intended system. It is* ***not*** *the final exam report and does not claim the split feature branches form a verified implementation.*

Sources compared: specification, `origin/main` `84846dc`, read `1ed58b6`, write `8c42a18`, update `59bbe80`, and election `d1222a2`.

1. How to reason about a distributed implementation

Reading classes one at a time is not enough. Correctness lives in connections:

- Who creates an ID?
- Which messages carry it?
- When does a value become visible?
- Which timeout takes responsibility after silence?
- What old messages can still arrive after a role change?
- What evidence survives a crash?

For every scenario, trace actor state, transaction state, message queues, timers, and replicated history separately.

2. Core invariants

Actor encapsulation

Each actor owns its mutable state. Messages crossing actors do not expose mutable arrays/lists.

Transaction routing

Every simultaneously active local FSM has a unique `TransactionId`. A message is processed only by its intended transaction and only when valid for the current state/generation.

Update identity

Every logical update has one stable, unique `EpochPair` carried consistently through proposal, acknowledgement, commit, history, and recovery.

At-most-once application

A replica applies each ordered update no more than once and only after valid commit evidence.

Total order

All correct replicas apply committed updates in ascending epoch/sequence order.

Uniform agreement

If any replica applies an update, every correct replica eventually applies it, even if the first replica later crashes.

Term transition

Old required work is recovered before new-coordinator updates begin.

Crash-stop behavior

After a replica becomes crashed, no queued or incoming event causes further protocol output or application progress.

3. Healthy write scenario

To argue a healthy write:

1. Client queue selects one request and allocates parent ID.
2. Contacted replica creates matching parent and distinct child update ID.
3. Coordinator serializes the child update against other writes.
4. Coordinator assigns next epoch pair before UPDATE.
5. Followers record exactly that proposal and ACK once.
6. Distinct ACK set reaches strict majority.
7. Coordinator sends WRITEOK in assigned order and applies once.
8. Followers apply once in same order.
9. Trigger child completes parent; parent returns result; client advances queue.

The feature branches currently implement pieces of this trace but not all invariants: stable pre-broadcast epoch assignment, distinct ACK tracking, actual position mutation on the update branch, and integrated serialization require attention.

4. Concurrent clients

Client A and B each serialize their own operations, but their requests can reach different replicas concurrently. The contacted replicas can forward child updates concurrently.

The coordinator must decide one order:

```
UA gets <e,7>, UB gets <e,8>
```

FIFO coordinator-to-follower channels then preserve that send order. If the coordinator runs both update FSMs without an ordering queue or stable assignment discipline, ACK timing can cause inconsistent commit order.

Therefore “actors process one message at a time” is helpful but insufficient: the coordinator may interleave messages for multiple active FSMs across mailbox turns.

5. Follower crash during normal update

With five replicas and quorum three, one follower can crash without blocking progress.

Safety reasoning:

- coordinator counts only itself plus distinct follower ACKs;
- WRITEOK is sent only after three witnesses;
- crashed follower applies nothing further;
- reliable channels deliver WRITEOK to surviving followers;
- recovery history retains the ordered update.

Liveness reasoning:

- coordinator does not wait for every follower;
- a surviving strict majority is sufficient.

The inspected partial update branch counts ACK messages rather than distinct sender IDs, so the intended argument needs a stronger concrete implementation.

6. Coordinator crashes before UPDATE

State:

- client waits on parent write;
- trigger replica owns child request;
- no follower has observed coordinator broadcast.

Responsibility transfers through the trigger's missing-UPDATE timeout. It starts/join election, waits for synchronization, then resubmits or reconstructs the request under the new coordinator.

Heartbeat may also detect failure, but the update-stage timeout gives a request-specific signal. The update branch's timeout-to-election/retry behavior is still commented out.

7. Coordinator crashes after UPDATE but before quorum

Some replicas have observed the update; none should apply without WRITEOK. Election candidates advertise freshness, enabling a replica with the newest observation to win.

Recovery must decide how to handle the partial proposal. The specification favors completing interrupted work to prevent losing a value that might have been delivered. An immutable history record is necessary to know `(index,value,epochPair)`.

A positions-only snapshot may not include an observed-but-unapplied update, so it cannot fully implement this reasoning.

8. Coordinator crashes after quorum or during WRITEOK

This is the critical uniform-agreement case. Some replicas may apply; others may only have observed.

The new coordinator must:

1. gather/possess the freshest ordered history;
2. detect the incomplete commit dissemination;
3. resend/complete the required update idempotently;
4. ensure correct survivors apply it once; and
5. only then begin the new epoch's normal writes.

Client timeout or coordinator death does not authorize rollback of an update already applied anywhere.

9. Silent coordinator crash

With no active updates, followers rely on heartbeat watchdogs.

False-election prevention requires:

- heartbeat interval and latency-aware watchdog budget;
- reset only for expected coordinator;
- cancellation plus watchdog generation;
- one election request after current expiry; and
- replacement of heartbeat FSM after synchronization.

Current heartbeat and election branches implement much of this handoff, though test coverage remains focused rather than exhaustive.

10. Next ring node crashes

Election sender forwards token and schedules ACK timeout tied to target and attempt version. On current timeout it marks target unavailable and chooses the next ring ID.

Progress measure: each failed attempt permanently adds one unavailable ID for that local election. Static finite membership and surviving majority prevent endless retry of the same neighbor.

Late ACK/timeout from a previous attempt must not clear or redirect current pending state.

11. Best election candidate crashes

After token collection, the best candidate may fail before accepting leadership. The election FSM tries to send the final list to the winner. If forwarding fails, it removes that candidate and recomputes from the remaining set.

The safety question is whether the removed candidate held unique required history. The majority assumption says at least one correct member should know the most recent update, but the concrete history/evidence representation must make that claim true.

12. Competing elections

Staggered starts reduce collisions. If two still exist, replicas choose one preferred initiator deterministically and reject the loser.

For convergence, every replica must apply the same preference rule and scope the comparison to the same failed coordinator. Completed-term sets reject later stale tokens.

Election transaction preference does not choose the final coordinator; it chooses which token performs candidate collection. Final candidate freshness still determines the winner.

13. Late old-term traffic

After synchronization, old heartbeat ticks, watchdog expiries, UPDATEs, ACKs, WRITEOKs, election timeouts, and tokens may remain queued.

Defenses include:

- transaction IDs scoped to old owner/term;
- removing old FSMs from active routing;
- strictly newer epoch validation;
- state and attempt/watchdog versions;
- completed-election sets; and
- sender/coordinator membership validation.

Every message type needs an explicit stale policy. FIFO cannot solve cross-sender or local-timer interleavings.

14. Read during update or recovery

Reads are local and can occur while an update is in transit. Before WRITEOK, a replica returns its old value. After applying WRITEOK, it returns the new value.

During recovery, a design choice is required:

- allow reads of current local state while writes are paused; or
- pause reads until synchronized.

The specification emphasizes pausing new writes; it does not clearly demand global read blocking. The implementation/report must state the chosen behavior and its consistency consequence.

15. Client uncertainty

A client may time out even when its update eventually commits. The provided API reports success or timeout but has no operation-status query.

This means the system is not providing exactly-once client command semantics across retries. If an application retries after timeout, duplicate suppression needs a stable client request identity. `TransactionId` could contribute, but only if retry rules preserve it and histories recognize it.

Do not confuse replica at-most-once application of one update ID with application-level exactly-once requests.

16. Current integration matrix

Implemented as separate components:

- actor/bootstrap and network infrastructure;
- transaction/message routing;
- client write wrapper;
- branch-local read FSM;
- partial branch-local update FSM;
- heartbeat FSM;
- election FSM and validated snapshot synchronization;
- test/static-analysis infrastructure.

Not established as one final verified system:

- merged read/update/election receive graph;
- stable update ordering assignment and serialization;
- complete positions/history mutation semantics;
- distinct ACK and duplicate handling;
- update-timeout election/retry;
- history-based recovery of incomplete updates;
- synchronization completion criterion; and
- end-to-end base/regression pass on the final integration commit.

17. How to prepare an exam trace

For each demonstration, draw five columns:

```
time | actor mailbox event | local FSM state | replicated state/history | outgoing messages
```

Mark the crash point exactly. Then explain:

1. which invariant is temporarily at risk;
2. which timeout or message transfers responsibility;
3. why stale events cannot undo the result; and
4. what external callback proves progress.

This method is more reliable than memorizing method names.

18. Final study checkpoint

Take this scenario: coordinator sent UPDATE to a quorum, sent WRITEOK to one follower, then crashed. Explain why electing any surviving replica by ID alone is insufficient. Your answer should mention observed history, uniform agreement, candidate freshness, idempotent completion, epoch transition, and delaying new writes until recovery.

The system-wide invariant to remember is: **one ordered update that becomes visible anywhere must remain recoverable and eventually become visible once, in the same order, at every correct replica.**

19. A student-friendly safety proof outline

```
Safety proof obligations
  1. one client request -> one parent transaction
  2. one update -> one ordered EpochPair
  3. ACK quorum -> one commit decision
  4. WRITEOK validation -> apply at most once
  5. recovery -> preserve committed/required history
  6. client queue -> later read cannot overtake earlier write
```

To argue sequential consistency, pick a sequential order for completed operations. Per-client order comes from the request queue. A write's position in the global order comes from its committed `EpochPair`. A read is placed after the writes visible at its target replica. The difficult step is proving that recovery never invents a second order for an update that was already visible. That proof needs the quorum, history, election, and synchronization reports together.

20. Corner-case matrix

Scenario	Safety question	Liveness question
duplicate ACK	can one sender count twice?	can a valid quorum still be reached?
coordinator crash after quorum	can a partial commit be lost/reordered?	do survivors elect and synchronize?
follower crash during WRITEOK	can it apply twice after restart?	can the system progress with a majority?
stale watchdog	can an old timer start a second election?	does a genuine silence still trigger one?
read during recovery	can it observe an unsafe snapshot?	when is the replica safe to serve?
client timeout then retry	can the write be duplicated?	is there an idempotency/reconciliation path?

Use this table in an exam answer to avoid discussing only the happy path. For each row, name the exact guard or state transition that answers the safety question and the message/timeout that answers liveness.

21. How to present a trace under time pressure

Write columns for actor, event, transaction ID, epoch, and visible state. Mark network hops with arrows and local timers with a clock symbol. At every event ask: "Could this be stale? Could this sender be wrong? Has the operation already completed?" This compact notation exposes most race bugs without reproducing the entire Java source.

Practice

Complete a trace for: client write, two ACKs in a five-replica system, coordinator crash, election, synchronization, then client read. Include one duplicate ACK and one stale watchdog. State exactly why each is ignored or accepted.